Introdução à GPU: Soma de Vetores em GPU

Kaiky S. Lima

1 - Ambiente

Componente	Propriedade	Especificação
Ambiente	Plataforma	Google Colaboratory (VM com GPU)
GPU	Modelo	[preencher: saída do !nvidia-smi]
GPU	Driver / versão do CUDA	[preencher: saída do !nvidia-smi]
Compilador	nvcc	[preencher: saída do !nvcc --version]
CPU	Modelo	[preencher: saída do !lscpu]
Memória RAM	Capacidade	[preencher: saída do !free -h]

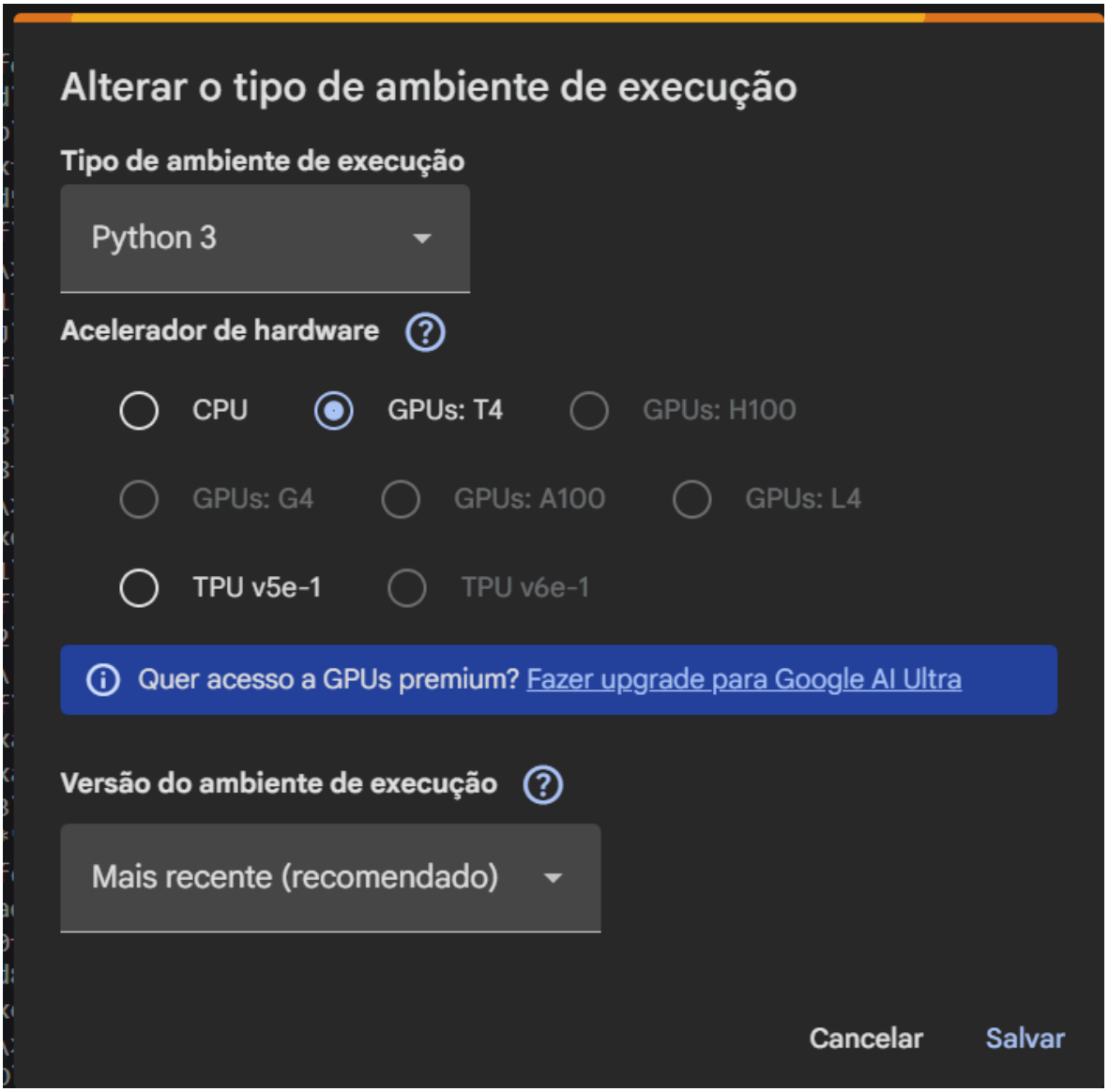
2 - Objetivo

Executar na GPU os exemplos hello-world-3 e VecAdd (soma de vetores) da Aula 13 e verificar como o arranjo de *threads* (grid de blocos) é criado e como o resultado é calculado e conferido.

3 - Modificações necessárias

Não foi alterado o código-fonte dos exemplos do `src.zip`. Apenas a compilação e a forma de execução:

- Alterar as configurações do ambiente do google para GPUs



4 - Resultados

Os dois exemplos executaram na GPU sem erros e o `VecAdd` produziu o resultado esperado nas duas execuções.

hello-world-3 (1 bloco × 1 *thread*)

```
!nvcc hello-world-3.cu -o hello-world-3.exe
!./hello-world-3.exe

Host: Hello World!!!
GPU: gridDim:( 1,  1,  1) blockDim:( 1,  1,  1) blockIdx:( 0,  0,  0) threadIdx:( 0,  0,  0) -> Thread[ 0]: Hello World!!!
```

VecAdd (256 *threads* por bloco, `(n + 255) / 256` blocos):

Elementos (n)	Memória nos 3 arrays (bytes)	Resultado final (soma, média)
20	240	(20,000000; 1,000000)
1.000.000	12.000.000	(1.000.000,000000; 1,000000)

Com n = 20 todos os `h_c[i]` foram 1,000000. Com n = 1.000.000, as primeiras e as últimas posições também foram 1,000000.